

epos4_controller の init 連射レースによる片輪が動かない問題

- 日付: 2026-06-04
- 環境: Linux ホスト / Docker コンテナ `rerobot_env` / ROS 2 Jazzy / `ros2_canopen`(CiA402Driver, CiA 402)、maxon EPOS4 x2 over can0(bitrate 1 Mbps, sync 50 ms)
- 対象ブランチ: `main` (HEAD: 33af0a0 feat: add debug report skill and template; enhance slam_toolbox configuration for message filtering)
- 関連ファイル:
 - `src/epos4_controller/src/epos4_controller.cpp`(修正対象)
 - `src/external/epos4compact50-5scan/maxon_epos4_ros2/config/bus_config_cia402_epos4_vel/bus.yml`(TPDO マッピング参照)
 - `src/epos4_teleop/src/teleop_keyboard.cpp`(症状の発生経路 — `/robot_speed_cmd` への Twist publish)

TL;DR

`teleop_keyboard` で wasd 操作したとき片輪(ユーザ機体では左、検証機体では右)が動かないことが頻発する問題。原因は `epos4_controller` コンストラクタが CiA402 遷移サービス `init/enable/cyclic_velocity_mode`(x2 モータ)を `async_send_request` で応答を待たずに連射し、かつコンストラクタが `rclcpp::spin()` 前に走るため応答 future を処理できず、前の遷移完了を確認しないまま次を撃つレースだったこと。失敗モータは `init` の Homing モード(mode=6)/Switch-On-Disabled(statusword `0x0240`)に取り残され、目標速度を受け付けない。約 4 回に 1 回再現し、**2 番目に初期化される側が落ちやすい**(=機体差で左右どちらにも出る)。修正は `init` を `spin` 中のワーカースレッドへ移し、各応答を future で待つ**逐次化**、最後に SDO で statusword/mode を**検証**して未達なら `recover`+リトライ。実機 10 回連続再起動で両モータとも attempt 1 成功(リトライ発動ゼロ)。

症状

- **いつ**: スタック起動(bus_config → epos4_controller)直後。 `teleop_keyboard` を起動し wasd で速度指令を与えたとき。
- **どこで**: 差動駆動の片輪。ユーザ報告では左輪(motor1)、検証機体では右輪(motor2)が頻繁に無反応。
- **何が起きるか**: 片方のモータだけ目標速度に追従せず回らない。 `teleop` / コントローラ側にエラーは出ず、指令(TPDO 目標速度)は両モータへ正しく送出されている。間欠的(毎回ではない)。
- **正常時**: `/robot_speed_cmd` の Twist に対し両モータが対称に回転すべき。

実測(修正前、コントローラ再起動 x8、`linear.x=0.6` を 2.3 s 指令して position(0x6064)差分の絶対値で回転を判定):

```

iter 1: dLEFT=-1.0707    dRIGHT=+0.0000    <- 右輪 無回転
iter 2: dLEFT=-1.7410    dRIGHT=+0.0000    <- 右輪 無回転
iter 3: dLEFT=NA         dRIGHT=NA         (サンプリング失敗)
iter 4: dLEFT=-1.6045    dRIGHT=+0.0000    <- 右輪 無回転
iter 5: dLEFT=NA         dRIGHT=NA         (サンプリング失敗)

```

```

iter 6: dLEFT=-1.5216    dRIGHT=NA
iter 7: dLEFT=-1.8054    dRIGHT=-1.7962    <- 両輪OK
iter 8: dLEFT=-1.9757    dRIGHT=+0.0000    <- 右輪 無回転

```

注: `joint_states.velocity` は常に 0(後述の通り `bus.yml` で 0x606C 未マップ)のため、回転判定には `position` 差分を使う必要がある。

切り分けの記録

1. teleop 側の左右非対称性 → 否定

`teleop_keyboard.cpp` は `/robot_speed_cmd` に Twist を publish するだけで左右対称。指令経路に非対称要素はなく、原因は下流(コントローラ or EPOS4 状態)と判断。

2. 指令(TPDO 目標速度)はモータへ届いているか → 届いている

孤立 teleop の干渉を疑ったが `/robot_speed_cmd` の Publisher count=0 で除外。 `linear.x=0.15` を指令しコントローラ出力 TPDO を直読み:

項目	観測値	備考
<code>/motor1/.../tpdo data</code>	4294967293 (=0xFFFFFFFFD = -3)	motor1 へ目標速度 -3 rpm
<code>/motor2/.../tpdo data</code>	4294967293 (= -3)	motor2 へも同値

→ コントローラは両モータへ対称に目標速度を送出している。問題は EPOS4 の受け側状態にある。

3. joint_states.velocity が常に 0 → 計測信号の誤り(罨)

当初 velocity で回転判定しようとしたが、健全時でも両モータ velocity=0.0。 `bus.yml` の TPDO1 マッピングで velocity actual value (0x606C) がコメントアウトされており、送信されるのは statusword(0x6041)/ position(0x6064)/ mode display(0x6061)のみ。回転判定は **position 差分**に切替(症状セクションの実測)。

4. 失敗モータの CiA402 状態を SDO 直読み → 当たり

`init`(homing 起動)後、cmd=0 のまま

`/motor{1,2}/.../sdo_read(canopen_interfaces/srv/C0Read)`で statusword(0x6041)/ mode display(0x6061)を再起動 ×4 で読取り:

iter	M1 statusword	M1 mode	M2 statusword	M2 mode	判定
1	0x1237	9	0x1237	9	両健全
2	0x1237	9	0x1237	9	両健全
3	0x1237	9	0x1237	9	両健全
4	0x1237	9	0x0240	6	M2 失敗

- 健全: statusword 0x1237(下位 0x37 & 0x6F = 0x27 = **Operation Enabled**)、mode 9(CSV)
- 失敗: statusword 0x0240(0x40 = **Switch On Disabled** = 未 enable)、mode 6(Homing = CSV へ未切替)

→ 失敗モータは init の Homing モードに取り残され、後続 `enable/cyclic_velocity_mode` が反映されていないと確定。

根本原因

旧 `epos4_controller.cpp` のコンストラクタは 6 つの遷移サービスを次のように発行していた:

```
call_trigger_service(m1_client_driver_init_, "init");          // motor1
call_trigger_service(m1_client_driver_enable_, "enable");
call_trigger_service(m1_client_driver_csv_mode_, "cyclic_velocity_mode");
call_trigger_service(m2_client_driver_init_, "init");          // motor2
call_trigger_service(m2_client_driver_enable_, "enable");
call_trigger_service(m2_client_driver_csv_mode_, "cyclic_velocity_mode");
```

`call_trigger_service` は内部で `async_send_request(req, callback)` を呼んで即 `return` する(撃ちっぱなし)。問題は 2 点:

1. **応答完了を待たずに次を発行:** `init` → `enable` → `csv` が前段の遷移完了を確認せず連続発行される。
CiA402 状態遷移は SDO 往復を伴い時間がかかるため、`init` の homing 実行中に `enable/cyclic_velocity_mode` が重なって取りこぼされる。
2. **応答を処理できない文脈:** コンストラクタは `main()` の `rclcpp::spin(node)` より前に実行される。`async_send_request` の応答 future は executor が spin して初めて処理されるため、コンストラクタ内では完了の検知も逐次化もできない。

結果、片方のモータが Homing(mode=6)/Switch-On-Disabled(statusword `0x0240`)に取り残され、目標速度を受け付けない=その輪が動かない。

なぜ「2 番目に初期化される側」が落ちやすいか: 6 サービスは数 μ s 間隔で同一 CAN バス・同一 master 上の 2 ドライバへ殺到する。先発(motor1)の遷移トラフィックが流れている最中に後発(motor2)の `enable/csv` が重なるため、後発側が取りこぼしやすい。機体により左右どちらが「2 番目」かは体感が変わるため、ユーザは左、検証機体では右が落ちた(同一現象)。

なぜ間欠的か: タイミング依存のレースのため、バス/ドライバ処理の揺らぎ次第で成功する回もある(検証 $\times 8$ で 7/8 が片輪失敗だが iter7 は両輪成功)。

なぜ顕在化を見落としやすいか: `bus_config` 後に十分待ってから手動でコントローラを起動するとレースが緩和され成功率が上がる。`rerobot_bringup.launch.py` の 5s `TimerAction` も「サービス広告待ち」は満たすが「遷移完了の逐次化」は保証しないため、間欠失敗は残る。

修正

`src/epos4_controller/src/epos4_controller.cpp`。init を spin 中に動くワーカースレッドへ移し、各サービス応答を future で待って逐次化、最後に SDO で statusword/mode を検証して未達なら `recover+リトライ`(最大 5 回)。要 `colcon build --symlink-install --packages-select epos4_controller` + ノード再起動。

コンストラクタの連射を撤去し、スレッド起動に置換:

```
// 旧: 6 サービスを撃ちっぱなしで連射
// call_trigger_service(m1_client_driver_init_, "init"); ... (×6)

// 新: spin 開始後に動くワークスレッドで逐次初期化
init_thread_ = std::thread(&Epos4_Control2_Node::run_init_sequence, this);
```

追加した中核ロジック(抜粋):

```
// 同期 Trigger 呼び出し: ワークスレッドで応答 future を待つ(executor スレッドではない)
bool call_trigger_sync(client, name, timeout = 5s) {
    if (!client->wait_for_service(2s)) { /* error */ return false; }
    auto future = client->
>async_send_request(std::make_shared<Trigger::Request>());
    if (future.wait_for(timeout) != std::future_status::ready) { /* warn */
return false; }
    return future.get()->success;
}

// SDO で statusword(0x6041)/mode(0x6061)を読み、Operation Enabled + CSV を検証
bool motor_ready(sdo_client, name) {
    auto sw = read_sdo(sdo_client, 0x6041);
    auto mode = read_sdo(sdo_client, 0x6061);
    if (!sw || !mode) return false;
    bool op_enabled = ((sw.value() & 0x6F) == 0x27);
    bool csv = (static_cast<int8_t>(mode.value()) == 9);
    return op_enabled && csv;
}

// 1 モータを CSV/Operation-Enabled へ。検証 NG なら recover してリトライ(最大5回)
void init_motor(name, init, enable, csv, recover, sdo) {
    for (int attempt = 1; attempt <= 5 && !stop_init_ && rclcpp::ok();
++attempt) {
        call_trigger_sync(init, name + " init"); // homing は失敗するが CSV では不要(想定内)
        call_trigger_sync(enable, name + " enable");
        call_trigger_sync(csv, name + " cyclic_velocity_mode");
        std::this_thread::sleep_for(200ms);
        if (motor_ready(sdo, name)) return; // 検証OK
        call_trigger_sync(recover, name + " recover");
        std::this_thread::sleep_for(300ms);
    }
    // 5回失敗 -> ERROR ログ(その輪は動かない)
}

// motor1 → motor2 を逐次。サービス広告を 20s 待ってから着手
void run_init_sequence() {
    if (!m1_client_driver_init_->wait_for_service(20s)) return;
    init_motor("motor1(left)", m1_..., m1_client_sdo_read_);
```

```
    init_motor("motor2(right)", m2_..., m2_client_sdo_read_);
}
```

付随変更: `sdo_read` クライアント(`canopen_interfaces/srv/CORead`)を `motor1/2` に追加、
`init_thread/std::atomic<bool> stop_init_` をメンバに追加、デストラクタで
`stop_init_=true` → `init_thread_.join()`。すべて `// claude` 注記付き。

なぜこの方法か

- **ワーカースレッド + 同期呼び出し**: コンストラクタは `spin` 前で応答を処理できないため、`init` を「`spin` 中に動く別スレッド」へ移すのが最小の構造変更で逐次化を実現できる。`main()` の `executor` が応答を捌き、スレッドは `future` を待つだけなので `executor` をブロックしない。`one-shot timer` 案は単一スレッド `executor` 内で `spin_until_future_complete` がデッドロックするため不採用。
- **SDO による検証 + リトライ**: 逐次化だけでもレースは消えるが、`homing` 由来の取り残しに対する保険として `statusword(0x27)/mode(9)` を実機の真値で確認し、未達なら `recover` してやり直す。検証で読み失敗(`nullopt`)時は `false` 扱い=リトライ側に倒すため、誤って「成功」と判定しない。
- **init を残す理由**: `homing` は CSV では不要で必ず失敗ログを出すが、`fault reset` 等の副作用がある可能性があり、安全側で残置。検証+リトライで吸収する。
- **破綻条件 / 再評価トリガー**: 5 回リトライしても CSV/Operation-Enabled に到達しない=ハード起因(配線/電源/フォルト)を疑う。`wait_for_service(20s)` は `device_manager boot` がこれより遅い構成変更時に見直す。

検証

実機 `rerobot_env`、`bus_config` 起動済みの状態でコントローラを 10 回再起動し、各回でコントローラ自身の in-process SDO 読みログ(`motor_ready`)で両モータの状態を確認:

項目	修正前	修正後
片輪が回らない頻度	~7/8 回(間欠)	0/10 回
両モータ Operation Enabled + CSV(0x1237/mode 9)到達	不安定	10/10 回、attempt 1
リトライ発動回数	(機能なし)	0(逐次化のみで成功、リトライは未使用)

修正後ログ抜粋(全 10 回同様):

```
[INFO] [epos4_controller_node]: motor1(left): statusword=0x1237 mode=9
(op_enabled=1 csv=1)
[INFO] [epos4_controller_node]: motor1(left) ready (Operation Enabled, CSV)
on attempt 1
[INFO] [epos4_controller_node]: motor2(right): statusword=0x1237 mode=9
(op_enabled=1 csv=1)
[INFO] [epos4_controller_node]: motor2(right) ready (Operation Enabled,
CSV) on attempt 1
[INFO] [epos4_controller_node]: EPOS4 auto-init complete.
```

検証手順(コンテナ内):

```
# 0. ビルド
colcon build --symlink-install --packages-select epos4_controller
source install/setup.bash

# 1. CAN & bus_config を起動(別シェル / バックグラウンド)
sudo ip link set can0 up type can bitrate 1000000 # 未upなら
ros2 launch maxon_epos4_ros2 bus_config_cia402_epos4_vel.launch.py

# 2. コントローラを起動し、auto-init ログで両モータの ready を確認
ros2 run epos4_controller epos4_controller \
  --ros-args --params-file src/rerobot_bringup/config/params.yaml
# -> "motor1(left) ready ... on attempt 1" と "motor2(right) ready ..."
の両方が出ること

# 3.(任意)実回転チェック: position 差分で判定(velocity は常に0なので不可)
#   ros2 topic pub -r30 /robot_speed_cmd geometry_msgs/msg/Twist "{linear:
{x: 0.6}}"
#   の前後で /motor{1,2}/.../joint_states の position[0] 差分が両輪とも非ゼロ
```

注意(検証時の罫): コントローラ起動直後に CLI から `ros2 service call .../sdo_read` を多数並走させると、100 Hz の PDO トラフィックと衝突して SDO 読みが空(→ statusword 0x0000/mode 0 とパースされる)になりやすい。これは測定アーティファクトであり、信頼源はコントローラ in-process のログ。

教訓 / 今後の予防

1. **CiA402(に限らず状態遷移)サービスは「撃ちっぱなし」にしない。** init→enable→mode は前段の応答を待ってから次を発行する。`async_send_request` の応答はノードが spin して初めて処理されるので、コンストラクタ内で発行した非同期要求の完了は当該コンストラクタ内では絶対に確認できない。初期化シーケンスはワーカースレッド or 起動後コールバックに置く。
2. 「成功した」を遷移サービスの戻り値だけで判断しない。実機の真値(statusword 0x6041 / mode 0x6061)を SDO で読んで `(sw & 0x6F)==0x27` かつ `mode==9` を確認するまで信用しない。
3. モータ回転の確認に `joint_states.velocity` を使わない。`bus.yaml` の TPDO で 0x606C(velocity actual)が未マップのため常に 0。 `position(0x6064)` 差分で判定する(`[[reference-joint-states-velocity-zero]]`)。
4. **docker / ROS デバッグでの `pkill` の罫**(本セッションで 2 回踏んだ):
 - `pkill -f "<pattern>"` は実行中シェル自身のコマンドラインにもマッチし自滅(exit 143)する。スクリプト本文にパターン文字列を含めない、または `pkill -x <comm>` (args ではなく comm 名照合)を使う。
 - `pkill -x <name>` の `<name>` は Linux の `comm`(プロセス名)が **15 文字に切り詰められる点**に注意。`epos4_controller`(16字)は一致せず、正しくは `epos4_controll`(15字)。これに気付かず各イテレーションのコントローラが 40+ 個累積した。
5. フォロアップ TODO(本件とは別バグ、未修正):
 - `epos4_controller.cpp` の `static_cast<int>(rpm)` による rpm 整数切り捨て。
gear_ratio=0.2 で低速指令はモータ rpm が小さく (0.15 m/s → -3.8 → -3 rpm)、さらに小さい指

令は 0 rpm に丸まって動かない量子化問題。

- `bus.yml` TPDO に 0x606C を追加して `joint_states.velocity` を有効化するか検討(現状は position 差分依存)。